一、路由层

二、教学目标

1、掌握静态文件的使用以及配置

2、理解HTTP请求体和响应

3、完成数据库的迁移

4、完成登录操作

三、静态文件

3.1 概述

网站通常需要提供类图片、`JavaScript` 或 `CSS` 的额外文件服务。在 `Django` 中，将这些文件称为静态文件。

在`Django` 在提供了 `django.contrib.staticfiles` 包来管理它们。

3.2 配置

第一步：确保 `INSTALLED_APPS` 包含了 `django.contrib.staticfiles`包

第二步：在配置文件中定义 `STATIC_URL`，该配置项指在访问静态文件时要使用的 `URL`，且必须以斜线结尾：

```
STATIC_URL = '/static/'
```

第三步：在配置文件中定义`STATICFILES_DIRS`，该配置项用于定义静态文件应用在启用 `FileSystemFinder` 查找器时将查找的额外位置，如：

```
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

需要注意的是：这些路径应该使用 `Unix` 风格的斜线，即使在 `Windows` 上也是如此。另外，如果指定了多个路径，在访问静态文件时将按路径的顺序依次查找，直到找到相关的静态文件为止（找到即停止）。

3.3 访问静态文件

访问静态文件时，可以在模板中，用 `static` 模板标签基于配置 `STATICFILES_STORAGE` 给定的相对路径构建 `URL`，示例代码如下：

1.导入bootstrap前端框架

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
     <!--使用最原始的方式-->
    <!-- 导入外部的css-->
    <!--<link rel="stylesheet" href="../static/bootstrap-3.4.1-
dist/css/bootstrap.min.css">-->
    <!-- 导入外部的js-->
    <!-- <script src="../static/js/jquery-3.7.0.min.js"></script>-->
    <!-- 导入外部的js-->
    <!--<script src="../static/bootstrap-3.4.1-dist/js/bootstrap.min.js">
</script>-->


    <!--使用激活静态资源的方式-->
    <!-- 导入外部的css-->
    <!-- <link rel="stylesheet" href="/static/bootstrap-3.4.1-
dist/css/bootstrap.min.css">-->
    <!-- 导入外部的js-->
    <!-- <script src="/static/js/jquery-3.7.0.min.js"></script>-->
    <!-- 导入外部的js-->
    <!--<script src="/static/bootstrap-3.4.1-dist/js/bootstrap.min.js"></script>-
->

    <!--使用模版语法的方式-->
    {% load static %}
     <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
     <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js"%}>
</script>


</head>
<body>
    <h1 class="text-center">我爱python</h1>
</body>
</html>
```

需要注意的是：`{% load static %}` 标签在独立的静态文件中（如 `CSS` 文件或 `JS` 文件）中是不可用的，因为它不是由 `Django` 生成的。应该始终使用 **相对路径** 在的静态文件之间相互引用。

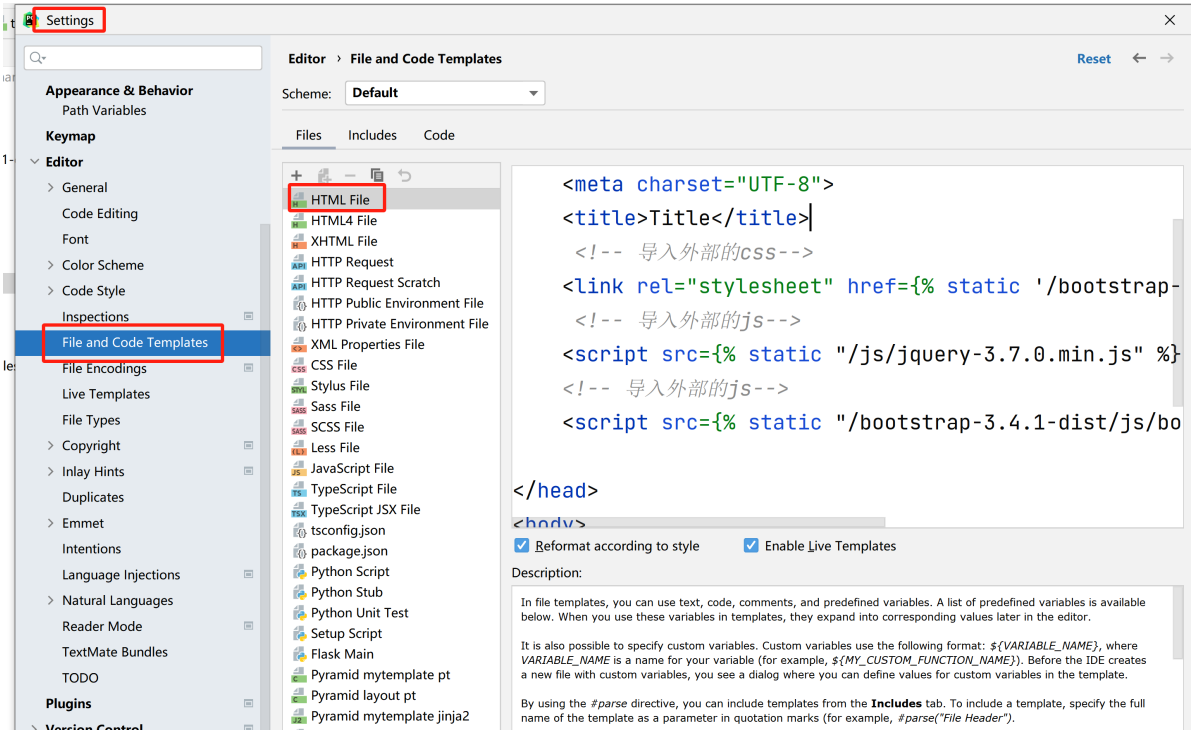
3.4 简捷方式

如果不想每次在模版中加载静态文件都使用 `load` 加载 `static` 标签，那么可以在 `settings.py` 中的 `TEMPLATES/OPTIONS` 添加 `builtins`，代码如下：

```
TEMPLATES = [
    {
        ...
        'OPTIONS': {
            'context_processors': [
                ...
            ],
            'builtins':[
                'django.templatetags.static'
            ]
        },
    },
]
```

这样以后在模版中就可以直接使用 `static` 标签，而无需手动加载。

技巧:为了防止每次新建的html文件都要引用bootstrap 的样式,可以将写好的 html代码写成模版文件

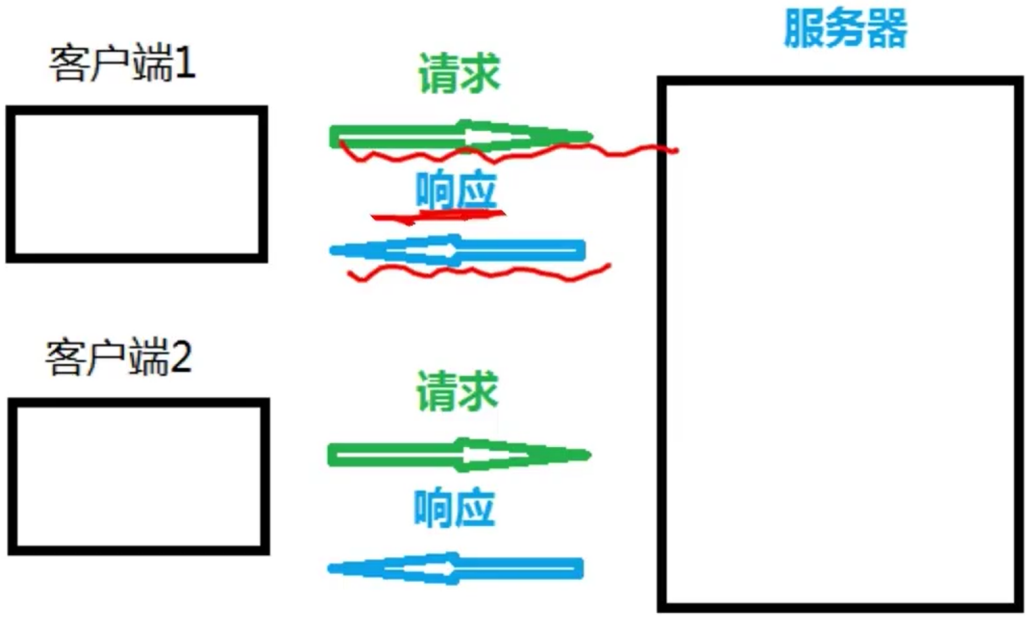


这样每次新建的都会有引用的代码

四、理解HTTP请求体以及响应状态码

4.1 请求定义:

4.1.1 请求是指浏览器端通过HTTP协议发送给服务器端的数据



浏览器F12 随便抓包一个数据

```
POST /v1/tokens HTTP/1.1      -> 起始行

Host: 127.0.0.1:8000          -> headers
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64;
rv:74.0) Gecko/20100101 Firefox/74.0
Accept: */*
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-
US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Content-Type: application/json
Content-Length: 58
Origin: http://127.0.0.1:7000
Connection: keep-alive
Referer:
http://127.0.0.1:7000/dadashop/templates/login.html

{"username":"guoxiaonao"}     -> body
```

4.1.2 请求中的方法:

- 根据HTTP标准，HTTP许求可以使用多种请求方去。
- 最常用HTTP1.0定义了三种请求方法:GET,POST 和HEAD方法(最常用)
- HTTP1.1新增了五种请习方法:OPTIONS, PUT,DELETE,TRACE和 CONNECT 方法。

序号	方法	描述
1	GET	请求指定的页面信息，并返回实体主体。
2	POST	向指定资源提交数据进行处理请求(例如提交表单或者上传文件)。数据被包含在请求体中。POST请求可能会导致新的资源的建立和/或已有资源的修改。
3	HEAD	类似于get请求，只不过返回的响应中没有具体的内容，用于获取报头
4	PUT	从客户端向服务器传送的数据取代指定的文档的内容
5	DELETE	请求服务器删除指定的页面。
6	CONNECT	HTTP/1.1协议中预留给能够将连接改为管道方式的代理服务器。
7	OPTIONS	允许客户端查看服务器的性能。
8	TRACE	回显服务器收到的请求，主要用于测试或诊断

4.1.3 Django当中的请求:

- 请求在Django中实则就是 视图函数的第一个参数，即HttpRequest对象
- Django接收到http协议的请求后，会根据请求数据报文创建HttpRequest对象
- HttpRequest对象 通过属性 描述了 请求的 所有相关信息
    - path info: URL字符串 method:字符串，表示HTTP请求方法，常用值:'GET、'POST'
    - GET:QueryDict查询字典的对象，包含get请求方式的所有数据
    - POST:QueryDict查询字典的对象，包含post请求方式的所有数据 FILES:类似于字典的对象，包含所有的上传文件信息

案例演示:

```
# 请求
urls.py:
path("request_test/", views.request_test),


views.py

def request_test(request):
    print("path信息",request.path_info)
    print("提交方式",request.method)
    print("使用GET请求查询提交数据", request.GET)
    #可以使用地址提交数据看看 127.0.0.1:8000/request_test?a=1&b=1
    print("使用GET请求查询提交数据", request.GET)

    return HttpResponse("OK")
```

还有其他的自行测试 现在先有个印象即可

- COOKIES:Python字典，包含所有的cookie，键和值都为字符串
- session:似于字典的对象，表示当前的会话
- body:字符串，请求体的内容(POST或PUT)
- scheme:请求协议('http'/'https')
- request.get_full_path():请求的完整路径
- request.META:请求中的元数据(消息头)
  -request.META['REMOTE ADDR']:客户端IP地址

4.2 响应定义:

响应是指服务器端接收到请求后做相应的处理后再回复给浏览器端的数据

4.2.1 HTTP状态码的英文为HTTP Status Code

下面是常见的HTTP状态码:

> 200-请求成功
> 301-永久重定向-资源(网页等)被永久转移到其它URL
> 302-临时重定向
> 404-请求的资源(网页等)不存在
> 500-内部服务器错误

4.2.2 django中响应对象

构造函数格式:
HttpResponse(contene=响应体，content_type=响应体数据类型，status=状态码)

作用:
向客户端浏览器返回响同时携带响应体内容

常用的Content-Type如下

> 'text/html'(默认的，html文件)
>
> 'text/plain'(纯文本)
>
> 'text/css'(css文件)
>
> 'text/javascript'(js文件)
>
> 'multipart/form-data'(文件提交)
>
> 'application/json'(json传输)
>
> 'application/xml'(xml文件)

4.2.3 HttpResponse 子类

类型	作用	状态码
HttpResponseRedirect	重定向	302
HttpResponseNotModified	未修改	304
HttpResponseBadRequest	错误请求	400
HttpResponseNotFound	没有对应的资源	404
HttpResponseForbidden	请求被禁止	403
HttpResponseServerError	服务器错误	500

案例演示:

```
urls.py
path("set404/", views.set404),

views.py

from django.http import HttpResponse,HttpResponseNotFound
def set404(request):
    return HttpResponseNotFound("页面不存在")
```

五、三、登录

5.1 在url.py添加路由代码

```
 #登录页面
    path('login/',views.login)
```

5.2 在views.py 添加实现

```
#登录
def login(request):
    return render(request,'login.html')
```

5.3 在tmplates文件夹新建login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static '/bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "/js/jquery-3.7.0.min.js" %}></script>
```

```
    <!-- 导入外部的js-->
    <script src={% static "/bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<h1 class="text-center">登录</h1>
<div class="row">
    <!--
        一行12份
    左右两边占4份 中间占8份 -->
    <div class="col-md-8 col-md-offset-2">
        <!--
            1. form 表单的 action 参数
            action="" -> 当前页面提交请求
            action="完整url" -> 指定的地址提交请求
            action='/后缀/' -> 本服务器的某个 url 地址提交请求.

            2. method 提交方式
                POST 安全 出现在参数
                GET 不安全 出现在地址栏
        -->
        <form action="" method="post">
            <p>用户名:<input type="text" name="username" class="form-control">
</p>
            <p>密码:<input type="password" name="password" class="form-control">
</p>
            <p>爱好:
                <input type="checkbox" name="hobby" value="0">跳
                <input type="checkbox" name="hobby" value="1">唱
                <input type="checkbox" name="hobby" value="2">rap
            </p>


            <input type="submit" class="btn btn-primary btn-block">

        </form>

    </div>
</div>
</body>
</html>
```

5.4 在settings.py 注释一行代码

```
# 当前项目已安装且被启用的中间件列表
MIDDLEWARE = [
    ....
    #只要是POST请求这个需要注释 因为这是需要校验 目前我们不需要 CSRF令牌。前端没有正确提供这个
令牌
    # 'django.middleware.csrf.CsrfViewMiddleware',
    ....
]
```

5.5 在views.py 添加实现

```
from django.http import HttpResponse
from django.shortcuts import render,redirect

#登录
def login(request):
    # request.method: 返回请求的方式 POST/GET
    #   POST:  通过参数传递
    #       request.POST:提交请求的普通键值对数据(不包含文件)
    #       request.POST.get():获取健对应值(列表)中的最后一个元素
    #       request.POST.getlist(): 直接获取键值对(列表)
    #  GET:  通过url传递
    #       request.GET.get():获取健对应值(列表)中的最后一个元素
    #       request.GET.getlist(): 直接获取键值对(列表)
    if request.method == 'POST':
        print(request.POST)
        username = request.POST.get('username')
        print(username)
        password = request.POST.get('password')
        print(password)
        hobby=request.POST.getlist('hobby')
        print(hobby)
    else:
        username = request.GET.get('username')
        password = request.GET.get('password')
        print(username,password)


    return render(request,'login.html')
```

5.6 连接数据库

5.6.1 创建数据库

```
create database django_test;
```

5.6.2 在setting.py 完善数据库连接

```
# 数据库配置
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'django_test',
        'USER': 'root',
        'PASSWORD':'xxxxx',
        'HOST': '127.0.0.1',
        'PORT':3306
    }
}
```

5.6.3 在app01应用中models.py创建模型类

在操作数据库的时候介绍一个全新的用法

django.orm(对象关系映射) 后面会有专门一个章节讲解

【假如你完全没学习过sql也可以很顺利的通过python的代码操作数据库】

类 -->表

对象 -->一行数据

对象属性 --> 字段

```
#继承models.Model
class User(models.Model):

    # id in primary key auto_increment 表示主键 自增长
    id = models.AutoField(primary_key=True)
    # 表示username是 varchar(24)
    username = models.CharField(max_length=24)
    # 表示password是 varchar(24)
    password = models.CharField(max_length=24)
```

5.6.3.1 迁移数据库

先安装mysqlclient

```
pip install mysqlclient
```

终端:

将操作记录保存到migrations文件夹中

5.6.3.2 生成记录

```
python manage.py makemigrations
```

`看到 - Create model User 就说明成功`

5.6.3.3 同步数据库

```
python manage.py migrate
```

```
Apply all migrations: admin, app01, auth, contenttypes, sessions
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying app01.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
```

```
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

这就说明成功 稍后查看数据库 app01.user表这张表就是我们迁移的表

`为了做区分 因为你的项目不一定只有一个应用所以默认会加app01`

迁移完成之后 django自带的10张表不能删除 这些表都是后期操作django必要的表

5.7 完成登录

回到views.py完善代码

```
#登录
def login(request):
    # request.method: 返回请求的方式 POST/GET
    #   POST:  通过参数传递
    #       request.POST:提交请求的普通键值对数据(不包含文件)
    #       request.POST.get():获取健对应值(列表)中的最后一个元素
    #       request.POST.getlist(): 直接获取键值对(列表)
    #  GET:  通过url传递
    #       request.GET.get():获取健对应值(列表)中的最后一个元素
    #       request.GET.getlist(): 直接获取键值对(列表)
    if request.method == 'POST':
        print(request.POST)
        username = request.POST.get('username')
        #print(username)
        password = request.POST.get('password')
        #print(password)

        #相当于我们写的 select * from user whwere username=username
        # 查询多个值 返回的是列表
        res = models.User.objects.filter(username=username)

        if res:
            #获取第一条记录
            user_obj = res[0]
            if user_obj.password == password:
                #说明有用户 密码也正确 返回登录成功
                return HttpResponse("登录成功")
            else:
                return HttpResponse("密码错误")

        else:
            return HttpResponse("用户不存在")


    #     hobby=request.POST.getlist('hobby')
    #     print(hobby)
    # else:
```

```
    #     username = request.GET.get('username')
    #     password = request.GET.get('password')
    #     print(username,password)


    return render(request,'login.html')
```

六、作业

6.1 完成本天的案例